AI ASSIGNMENT

General-Purpose AI Reasoning System

PROJECT REPORT

Submitted by

Junaid Ahmed

Roll No.: M26MER001

Submission Date: 31 October 2026

1. Introduction

The purpose of this assignment is to build a program that can answer questions using a collection of facts and rules. The program reads the given information, converts supported sentences into a simple symbolic form, applies logical rules, and produces a **True** or **False** answer.

2. How the Program Works

First, the program separates the knowledge base into individual sentences. It identifies which sentences are facts and which are rules, and stores them in Python data structures. For example, “Bob is young” is stored as a predicate with Bob as its subject. A rule such as “If someone is young, then they are smart” is stored using a variable. The program then applies the rules to find conclusions that follow from the available information.

3. Reasoning Algorithm

I used **forward chaining** because it is a straightforward way to derive new facts from known information. The algorithm starts with the facts given in the question. It checks the conditions of each rule and adds the conclusion whenever all the conditions match. This process repeats until no new fact can be added. Finally, the program checks whether the requested statement is present in the derived facts.

4. Counterfactual Reasoning

I also included a basic causal-graph example for CounterBench. Statements such as “A causes B” are treated as directed connections between events. When a cause is changed as part of a hypothetical situation, the program removes its outgoing influence and checks whether the target event can still be reached. This demonstrates the basic idea for simple deterministic causal chains. It is not a complete solution for every type of counterfactual question.

5. Dataset and Evaluation

The assignment asks for RuleTaker and CounterBench to be used during development. The notebook includes code to attempt loading the public RuleTaker dataset and provides an evaluation helper. Dataset column names and formats should be checked after loading because they can differ between configurations. Accuracy is calculated by comparing the program’s predictions with the public labels. The program also measures execution time.

6. Results and Reasoning Trace

The notebook includes an example in which Bob is young and kind. Based on the rules, the system can derive that Bob is smart and successful. The program also displays the new facts it derives during reasoning. These steps make it easier to understand how the final answer was reached and to check the system when an answer is unexpected.

7. Limitations

The parser supports common sentence patterns used in the examples, but it cannot understand every possible English sentence. More complicated relations, quantifiers, and causal structures would need additional parsing and reasoning logic. The counterfactual part is also limited to a simple graph pattern. These limitations should be kept in mind when evaluating the system on the complete datasets.

8. Conclusion

This project demonstrates how classical AI reasoning can be implemented in Python. It converts supported natural-language facts and rules into a symbolic form, applies forward chaining, and checks whether a query can be derived. The system provides a clear starting point that can be improved with a stronger language parser and more advanced reasoning methods.

9. References

1. RuleTaker Dataset: https://huggingface.co/datasets/tasksource/ruletaker

2. CounterBench Dataset: https://huggingface.co/datasets/CounterBench/CounterBench